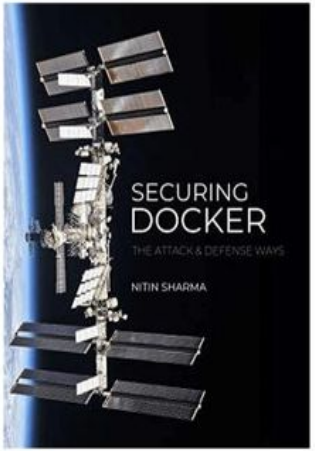
Practical Deep Dive into Container Internals and Forensics

Nitin Sharma
Lead Security Engineer, Salesforce

$ whoami

- Interested in AI, Threat Modeling, Security Research and Cloud Security Automation.
- Published Author for “Securing Docker - Attack and Defense Ways”



- Speaker at multiple technology conferences including AWS South Asia Conference 2021, GCP Cloud Community Days - Delhi, Defcon DC9111, Mule Dreamin, Bharat Dreamin, TDX Bengaluru, etc.
- Avid reader of personal finance books and a fitness enthusiast.

Container Forensics 101: What, Why and When

What:

Collecting, analyzing, and preserving evidence from compromised containers to understand attack vectors and timeline

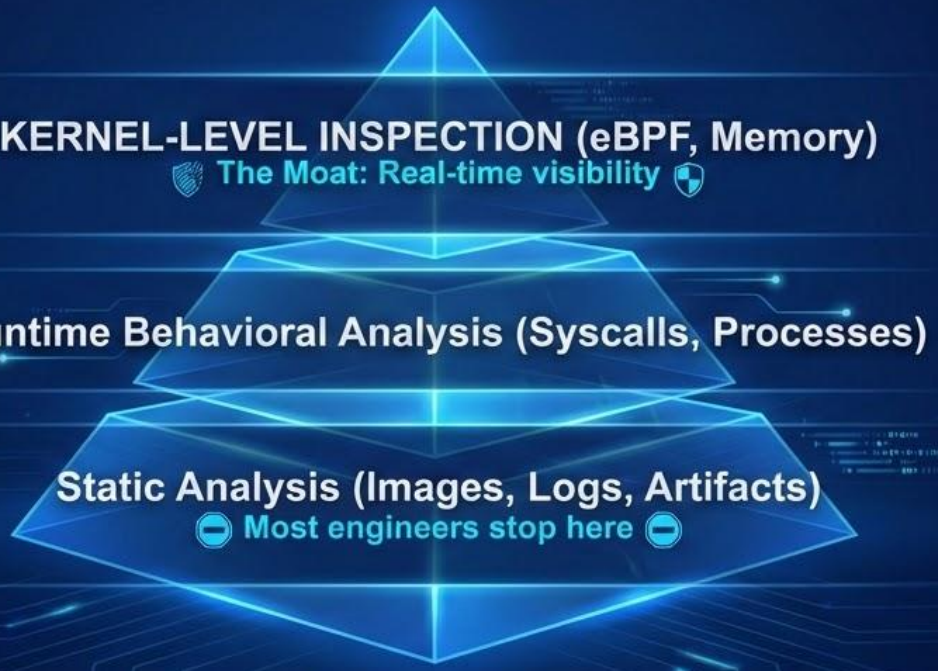
Why:

Containers are ephemeral (ghost like) - evidence disappears.

When:

After detection of suspicious activity, lateral movement, or data exfiltration

The Forensics Pyramid: How



Differentiator #1: eBPF Runtime Monitoring

Falco: Traces syscalls with pod/container context in real-time

Detection: Shell spawns in containers, unexpected network calls, sensitive file writes

Advantage: Zero-copy kernel visibility – nothing bypasses this layer

---

Most engineers: "Check logs." Top 10%: "Check what the kernel saw."

Differentiator #2: Behavioral Anomaly Detection

Process patterns

Web servers spawning shells (abnormal), unexpected privilege escalations

Namespace violations

Processes breaking out of container isolation, accessing host resources

Setuid abuse

Binaries with elevated permissions being executed by unprivileged users

Correlation with pod metadata = surgical threat detection

Differentiator #3: Live Memory & Post-Mortem Analysis

Memory dumps

Capture volatile artifacts via mechanisms before container dies

Filesystem snapshots

to preserve state

Chain of custody

Document acquisition timestamps, hash artifacts, version preservation

Container killed by attacker? <u>You already have the memory dump.</u>

Hands-On: Demo Scenario

⚠️ **INCIDENT ALERT**

Your monitoring detected a suspicious shell spawned inside a web-app container. You have 2 minutes before the container is destroyed.

What you'll do:

- Identify the container and extract critical artifacts
- Analyze process hierarchy and network connections
- Determine attack vector and impact scope

The 20% That Makes - Top 10% DFIR Engineers

- **Syscall tracing** > log analysis - Know what eBPF reveals
- **Metadata correlation** - Pod name + syscall = forensic gold
- **Chain of custody ritual** - Hash every artifact, document timestamps
- **Automate reconstruction** - Script log correlation, don't manually grep

You now know what separates incident responders from forensics experts.

Q&A + Next Steps

- Play with…
  - Falco (eBPF monitoring)
  - docker inspect / docker export
  - cri-tools (CRI forensics)
  - Sysdig and Falco (advanced tracing)

References

- https://github.com/docker-forensics-toolkit/toolkit
- https://github.com/google/docker-explorer
- https://medium.com/@deepanshu_khanna/docker-forensics-threat-hunting-like-a-pro-inside-containers-525d776cd8cf
- https://github.com/draios/sysdig
- https://github.com/abiosoft/colima